Artificial Neural Network

Refer to Notes on

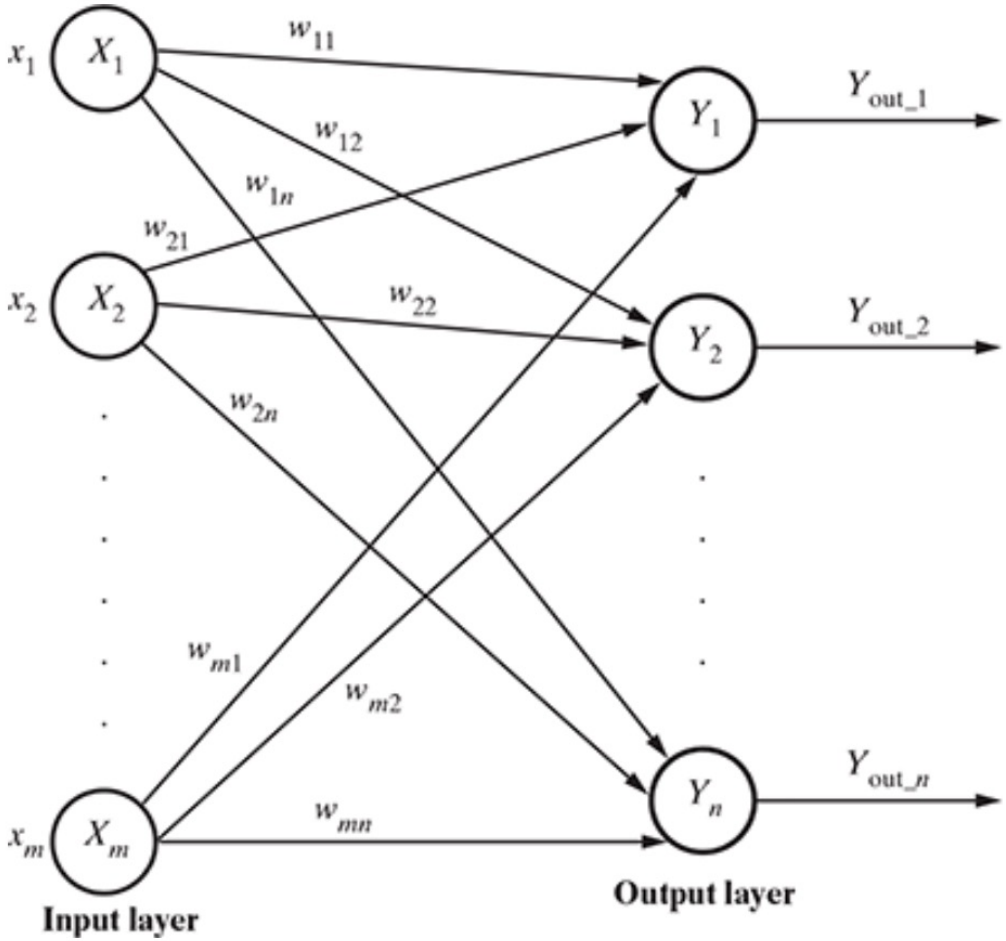
Architecture of Neural Network

- Artificial Neural Networks (ANNs) are computational models inspired by the human brain, consisting of interconnected artificial neurons.
- The architecture of a neural network determines how these neurons are organized and connected.
- The simplest form is the single-layer feed forward network, where input neurons pass signals directly to output neurons, suitable for basic classification tasks.
- More complex architectures include multi-layer feed forward networks, which introduce hidden layers to capture non-linear relationships.
- Competitive networks allow output neurons to compete, useful for clustering, while recurrent networks incorporate feedback loops, enabling the processing of sequential data.
- The choice of architecture depends on the problem’s complexity and data type.

Single-Layer and Multi-Layer Networks

- Single-layer feed forward networks consist of an input layer and an output layer, with signals flowing in one direction.
- These networks are easy to implement but limited in their ability to model complex data.
- Multi-layer feed forward networks, on the other hand, introduce one or more hidden layers between the input and output layers.
- Each layer can have a different number of neurons, and the connections between layers are weighted.
- This structure allows the network to learn more complex patterns and functions.
- Multi-layer networks are the foundation for most modern neural network applications, including image and speech recognition.

Single-Layer and Multi-Layer Networks



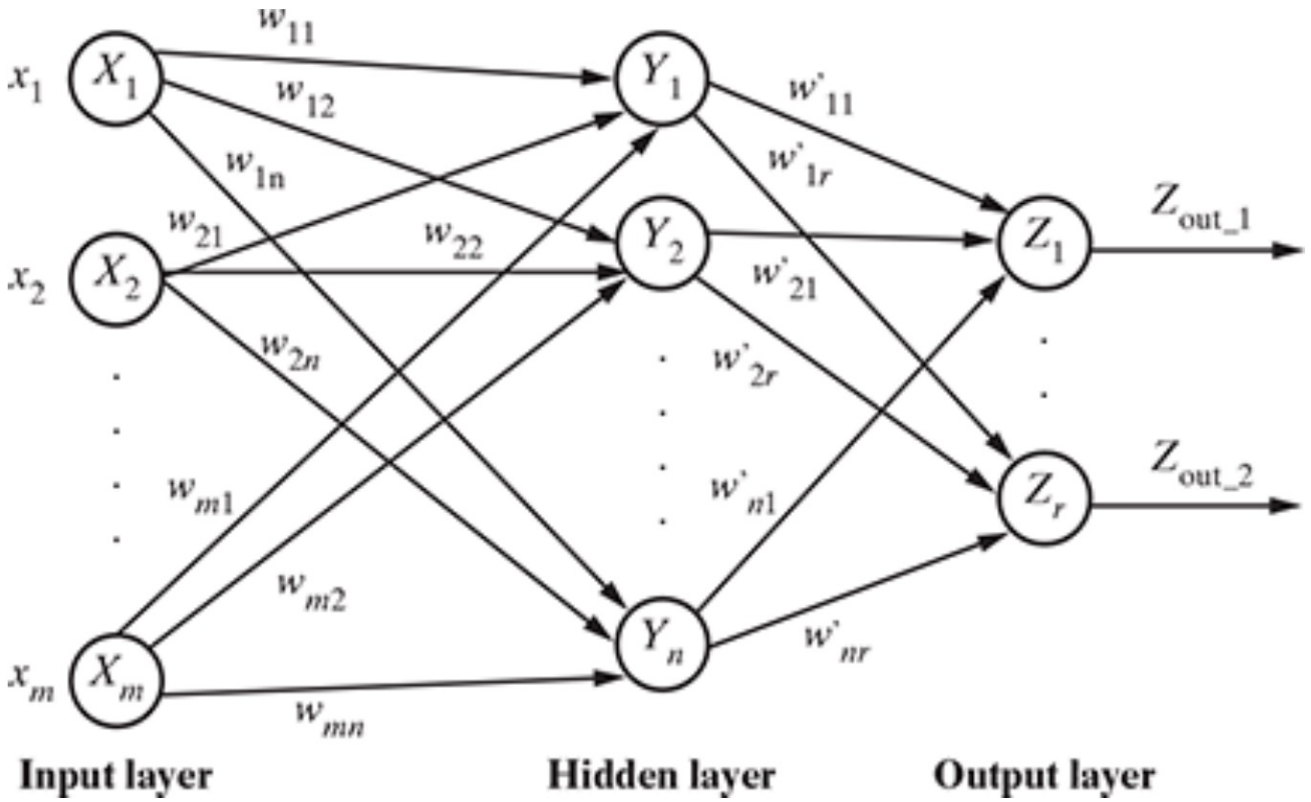
So, though it has two layers of neurons, only one layer is performing the computation.
This is the reason why the network is known as single layer in spite of having two layers of neurons.

Also, the signals always flow from the input layer to the output layer. Hence, this network is known as feed forward.

The net signal input to the k-th output neuron.

$$y_{in\_k} + x_1 w_{1k} + x_2 w_{2k} + \ldots + x_m w_{mk} = \sum_{i=1}^{m} x_i w_{ik},$$

Single-Layer and Multi-Layer Networks



The net signal input to the neuron in the hidden layer is given by

$$y_{in\_k} = x_1 w_{1k} + x_2 w_{2k} + \ldots + x_m w_{mk} = \sum_{i=1}^{m} x_i w_{ik},$$

for the *k*-th hidden layer neuron. The net signal input to the neuron in the output layer is given by

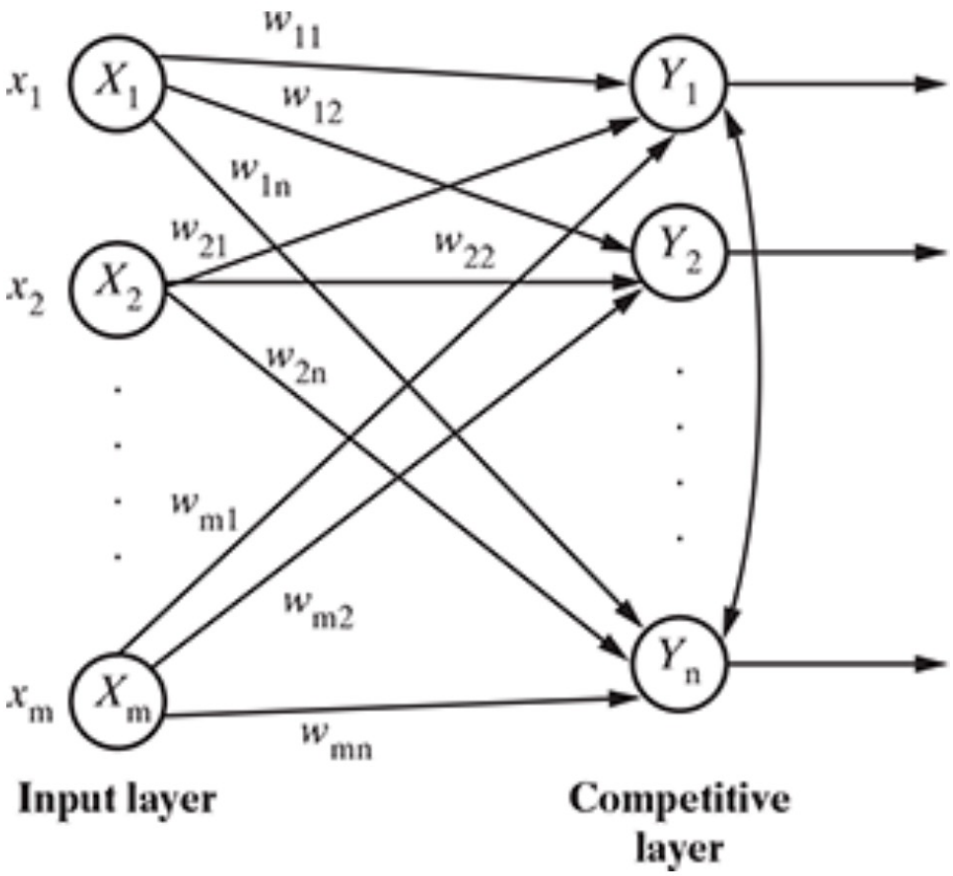
$$z_{in\_k} = y_{out\_1} w'_{1k} + y_{out\_2} w'_{2k} + \ldots + y_{out\_n} w'_{nk} = \sum_{i=1}^{n} y_{out\_i} w'_{ik},$$

for the *k*-th output layer neuron.

Competitive and Recurrent Networks

- Competitive networks are designed for unsupervised learning tasks, such as clustering.
- In these networks, output neurons are interconnected and compete to represent the input data, with only one neuron typically becoming active for a given input.
- Recurrent neural networks (RNNs) differ from feed forward networks by including feedback loops, allowing signals to travel from output neurons back to input neurons or within the same layer.
- This feedback mechanism enables RNNs to process sequential and time-dependent data, making them ideal for tasks like language modeling and time series prediction.
- Both architectures expand the capabilities of neural networks beyond simple classification

Competitive Networks

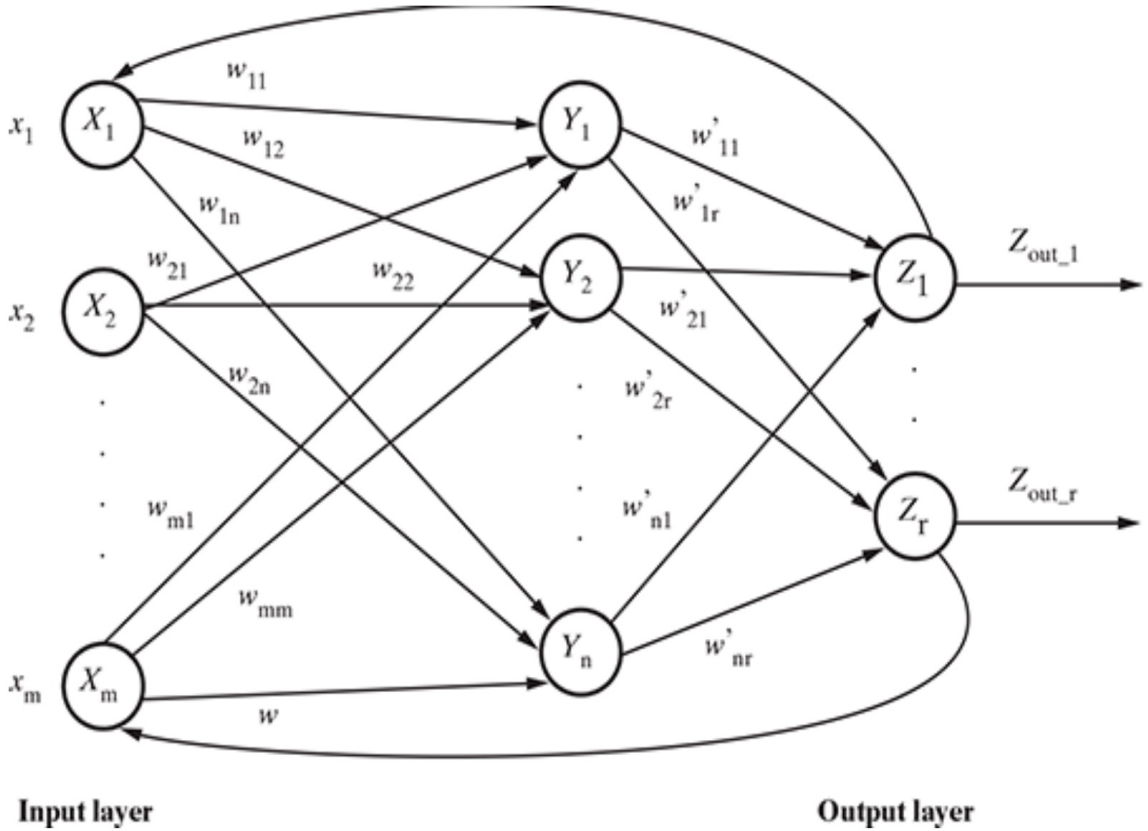


The competitive network is almost the same in structure as the single-layer feed forward network.

The only difference is that the output neurons are connected with each other (either partially or fully)

In competitive networks, for a given input, the output neurons compete amongst themselves to represent the input.

Recurrent network



We have seen that in feed forward networks, signals always flow from the input layer towards the output layer (through the hidden layers in the case of multi-layer feed forward networks), i.e. in one direction.

In the case of recurrent neural networks, there is a small deviation.

There is a feedback loop, from the neurons in the output layer to the input layer neurons.

There may also be self-loops.

Learning Process in ANN

- Learning in artificial neural networks involves adjusting the weights of connections between neurons to minimize prediction errors.
- The process starts with selecting the number of layers, the direction of signal flow, and the number of nodes in each layer.
- The weights are initialized, often randomly, and then iteratively updated during training.
- The learning process can be supervised, where the network is trained using labeled data, or unsupervised, where it discovers patterns without explicit labels.
- Stopping criteria, such as reaching a low error rate or a maximum number of iterations, are used to determine when training is complete.
- Proper learning ensures the network generalizes well to new data.

Learning Process in ANN

Learning in ANN involves adjusting weights to minimize prediction error.

Four main aspects determine how a network learns:

1. Number of Layers:
   - Single-layer (simpler tasks) or multi-layer (complex patterns).
2. Direction of Signal Flow:
   - Feed forward (unidirectional flow) or recurrent (feedback present).
3. Number of Nodes in Each Layer:
   - Input nodes = number of features.
   - Output nodes = number of classes or regression targets.
   - Hidden layer nodes are chosen manually or tuned experimentally.
4. Interconnection Weights:
   - Weights determine how signals are transmitted.
   - Must be iteratively updated to optimize learning.
5. Stopping criteria:
   Training stops when misclassification rate < threshold or max iterations reached.

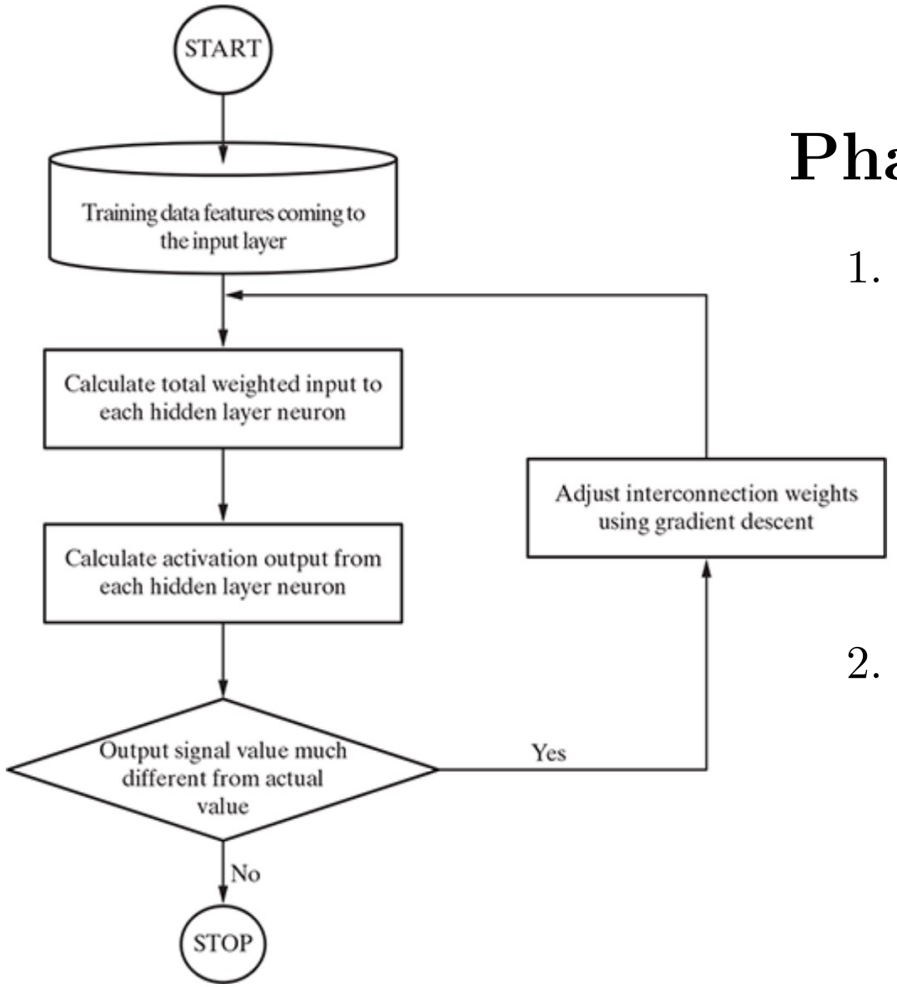
Weight Adjustment and Overfitting

- The most challenging aspect of training an ANN is determining the optimal weights for each connection.
- Initially, weights are set randomly, and through multiple iterations, they are adjusted to reduce the error between predicted and actual outputs.
- Overfitting can occur if the network has too many nodes or layers, causing it to memorize the training data rather than generalize.
- Techniques such as regularization, dropout, and early stopping are used to prevent overfitting.
- The balance between model complexity and generalization is crucial for effective learning.
- Monitoring the error rate and validation performance helps in tuning the network for best results.

Back-Propagation Algorithm

- Backpropagation is the most widely used algorithm for training multi-layer neural networks.
- It operates in two phases: the forward phase, where inputs are propagated through the network to generate outputs, and the backward phase, where errors are calculated and propagated back to update the weights.
- The algorithm uses gradient descent to minimize the cost function, which measures the difference between predicted and target outputs.
- The learning rate controls the size of weight updates, affecting convergence speed and stability.
- Backpropagation enables neural networks to learn complex mappings from input to output, making it essential for deep learning applications.

Back-Propagation Algorithm



Phases of Backpropagation

1. **Forward Phase:**
   - Inputs propagate through the network.
   - Each neuron computes weighted sums and applies activation functions.
   - Produces actual outputs.
2. **Backward Phase:**
   - Compute error $E = \frac{1}{2}\sum(t-o)^2$ where t = target output, o = actual output.
   - Propagate the error backward.
   - Update weights to minimize E.

Back-Propagation Algorithm

Weight Update Rule

Using **Gradient Descent**, weight adjustment for weight w_{jk} is given by:

$$\Delta w_{jk} = -\eta \frac{\partial E}{\partial w_{jk}}$$

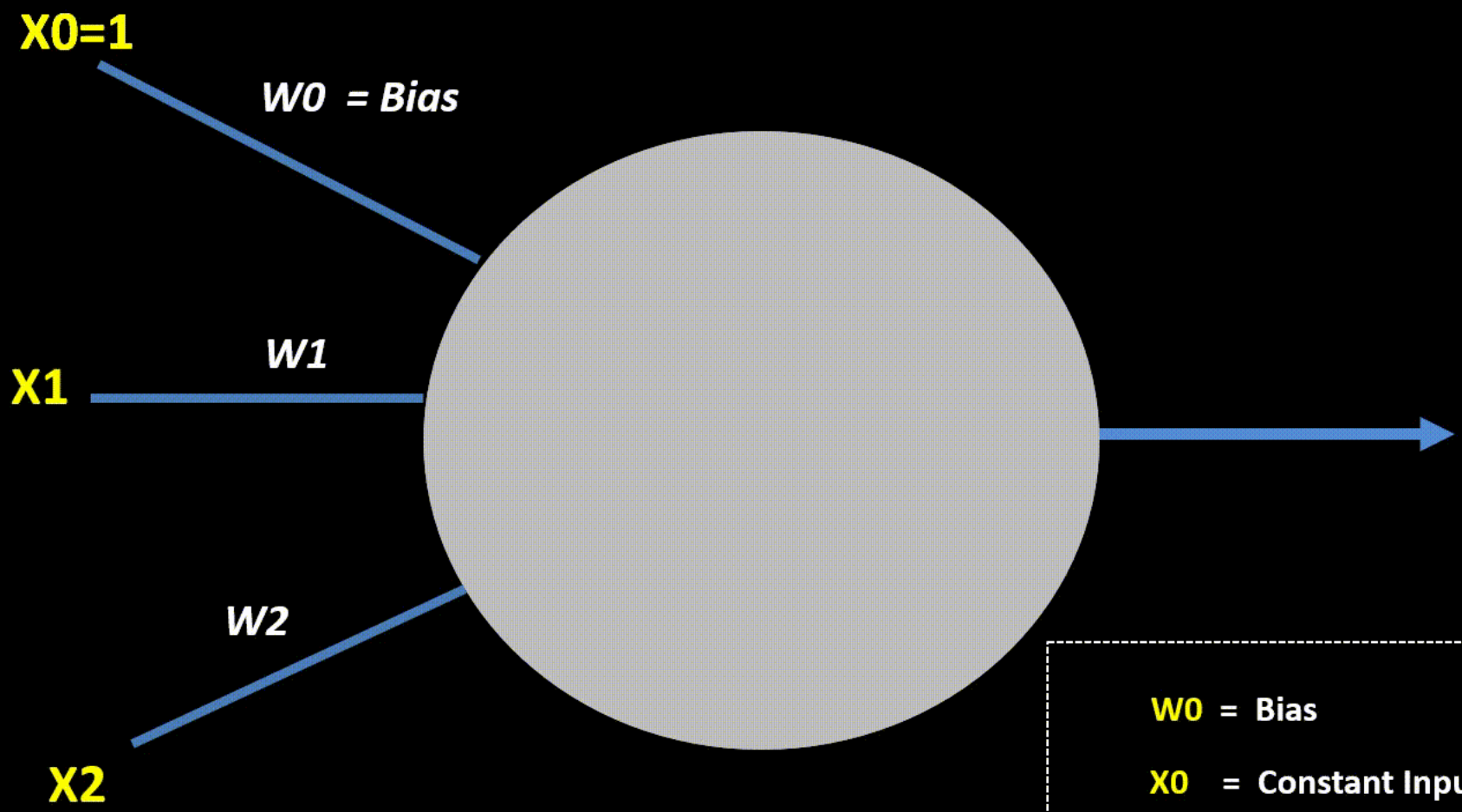
$$w_{jk}^{new} = w_{jk}^{old} + \Delta w_{jk}$$

where $\eta =$ **learning rate**.

Note on Learning Rate:

- High η: Faster learning but may overshoot minima.
- Low η: Slow but stable convergence.
- Often decreased gradually during training.

Artificial Neuron



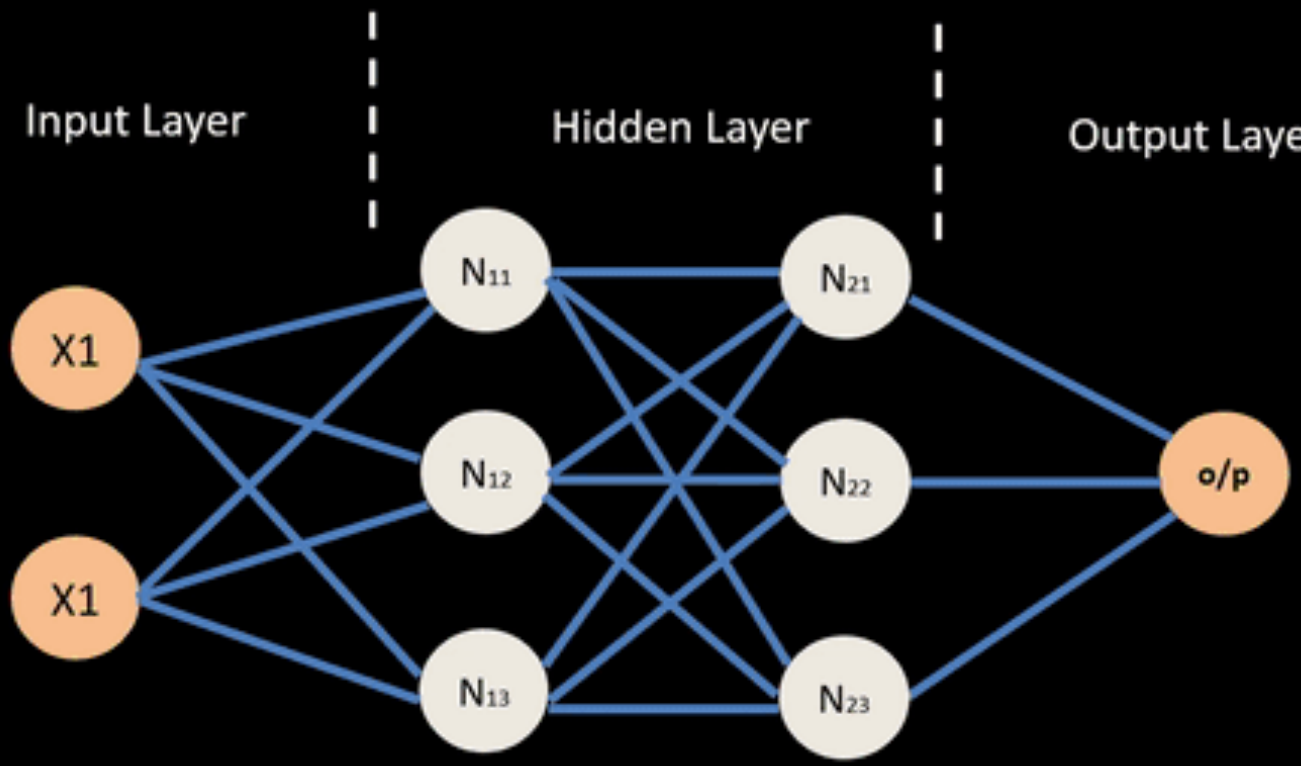
W0 = Bias

X0 = Constant Input 1 for Bias

X1,X2 = Attribute Inputs

W1,W2 = Weights of Inputs X1,X2

Neural Network – Backpropagation

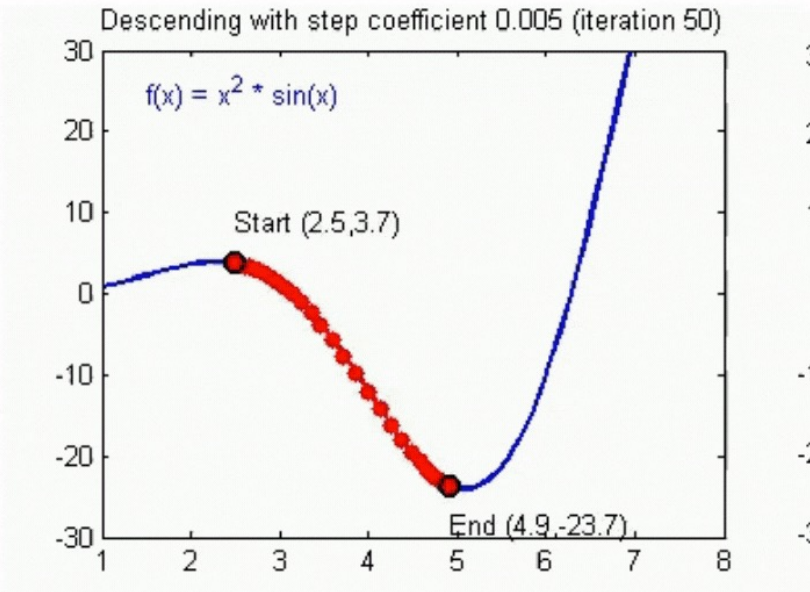


Gradient Descent and Learning Rate

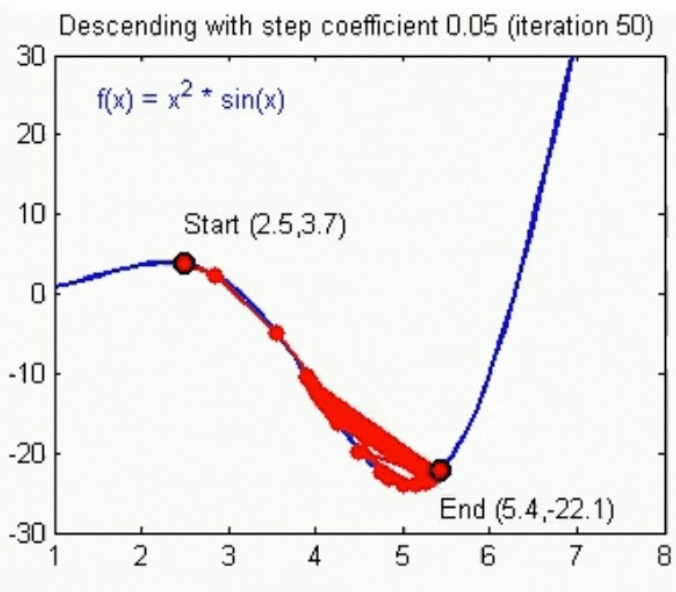
- Gradient descent is a mathematical optimization technique used in backpropagation to update the weights of a neural network.
- It calculates the gradient, or slope, of the cost function with respect to each weight and adjusts the weights in the direction that reduces the error.
- The learning rate determines how much the weights are changed during each iteration.
- A high learning rate can cause the algorithm to overshoot the minimum error, while a low rate can slow down convergence.
- Adaptive learning rates and techniques like stochastic gradient descent help improve training efficiency and accuracy, especially in large networks.

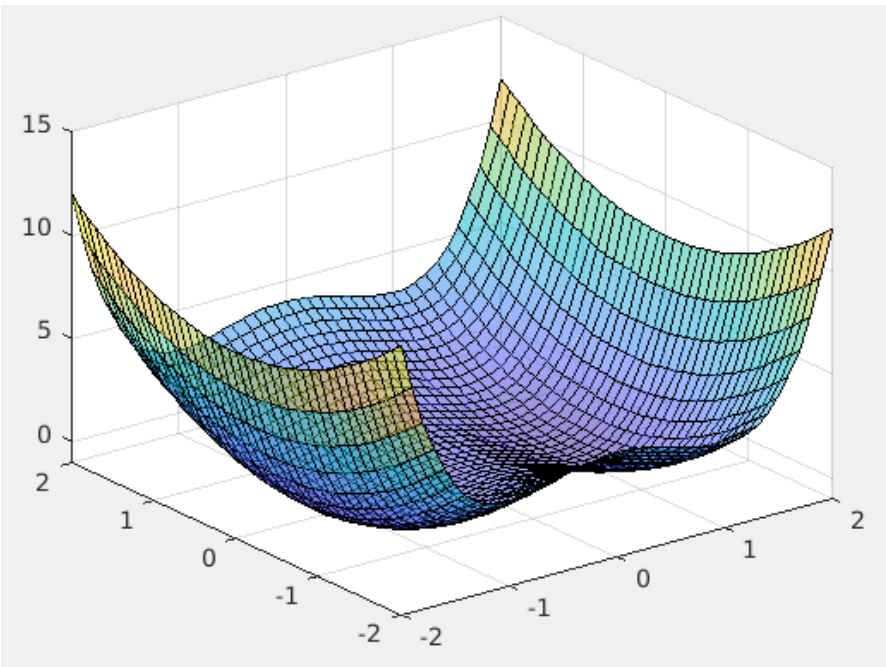
Gradient Descent and Learning Rate

Convergence



Divergence





Deep Learning

- Deep learning refers to neural networks with multiple hidden layers, known as deep neural networks (DNNs).
- These networks can learn hierarchical representations of data, with each layer extracting increasingly complex features.
- Deep learning has revolutionized fields such as computer vision, natural language processing, and speech recognition.
- Training deep networks requires significant computational resources, often utilizing GPUs for acceleration.
- Techniques like batch normalization, dropout, and advanced optimizers are used to improve training and prevent overfitting.
- Deep learning models have achieved state-of-the-art results in many applications, demonstrating the power of multi-layer architectures.